

Bayesian Linear Regression Simulation & Analysis (Honors Contract: PSTAT 115)

Kacie Chong

2025-11-09

```
# Load necessary packages
```

```
library(WDI)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    4.0.0      v tibble    3.2.1
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.0.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(broom)
library(ggplot2)
library(rstanarm)
```

```
## Loading required package: Rcpp
## This is rstanarm version 2.32.2
## - See https://mc-stan.org/rstanarm/articles/priors for changes to default priors!
## - Default priors may change, so it's safest to specify priors, even if equivalent to the defaults.
## - For execution on a local, multicore CPU with excess RAM we recommend calling
##   options(mc.cores = parallel::detectCores())
```

```
library(tidybayes)
library(bayesplot)
```

```
## This is bayesplot version 1.14.0
## - Online documentation and vignettes at mc-stan.org/bayesplot
## - bayesplot theme set to bayesplot::theme_default()
##   * Does _not_ affect other ggplot2 plots
##   * See ?bayesplot_theme_set for details on theme setting
```

```

library(bayesrules)

# Load World Bank indicators 2015-2020
## Internet users (%), Secondary school enrollment (%), GDP per capita (USD), Life expectancy at birth

raw <- WDI(
  indicator = c(
    "IT.NET.USER.ZS",
    "SE.SEC.ENRR",
    "NY.GDP.PCAP.CD",
    "SP.DYN.LE00.IN"
  ),
  start = 2015, end = 2020,
  extra = TRUE
)

# Clean and summarize data

cleaned <- raw %>%
  # kept rows only w/ all required indicators
  filter(!is.na(IT.NET.USER.ZS),
         !is.na(SE.SEC.ENRR),
         !is.na(NY.GDP.PCAP.CD),
         !is.na(SP.DYN.LE00.IN)) %>%
  # averaged values over 2015-20 per country
  group_by(country, region, income) %>%
  summarize(
    internet = mean(IT.NET.USER.ZS, na.rm = TRUE),
    school_enroll = mean(SE.SEC.ENRR, na.rm = TRUE),
    gdp = mean(NY.GDP.PCAP.CD, na.rm = TRUE),
    lifeexp = mean(SP.DYN.LE00.IN, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  # removed remaining NAs + transform variables
  filter(!is.na(internet), !is.na(school_enroll), !is.na(lifeexp)) %>%
  mutate(
    log_gdp = log(gdp + 1),
    internet_c = internet - mean(internet),
    school_c = school_enroll - mean(school_enroll),
    log_gdp_c = log_gdp - mean(log_gdp)
  )

summary(cleaned)

```

```

##      country      region      income      internet
## Length:202      Length:202      Length:202      Min.   : 3.00
## Class :character Class :character Class :character 1st Qu.:28.21
## Mode  :character Mode  :character Mode  :character Median :59.60
##                                     Mean  :55.60
##                                     3rd Qu.:78.31
##                                     Max.  :99.20
## school_enroll      gdp      lifeexp      log_gdp
## Min.   : 11.53      Min.   : 233      Min.   :39.76      Min.   : 5.455

```

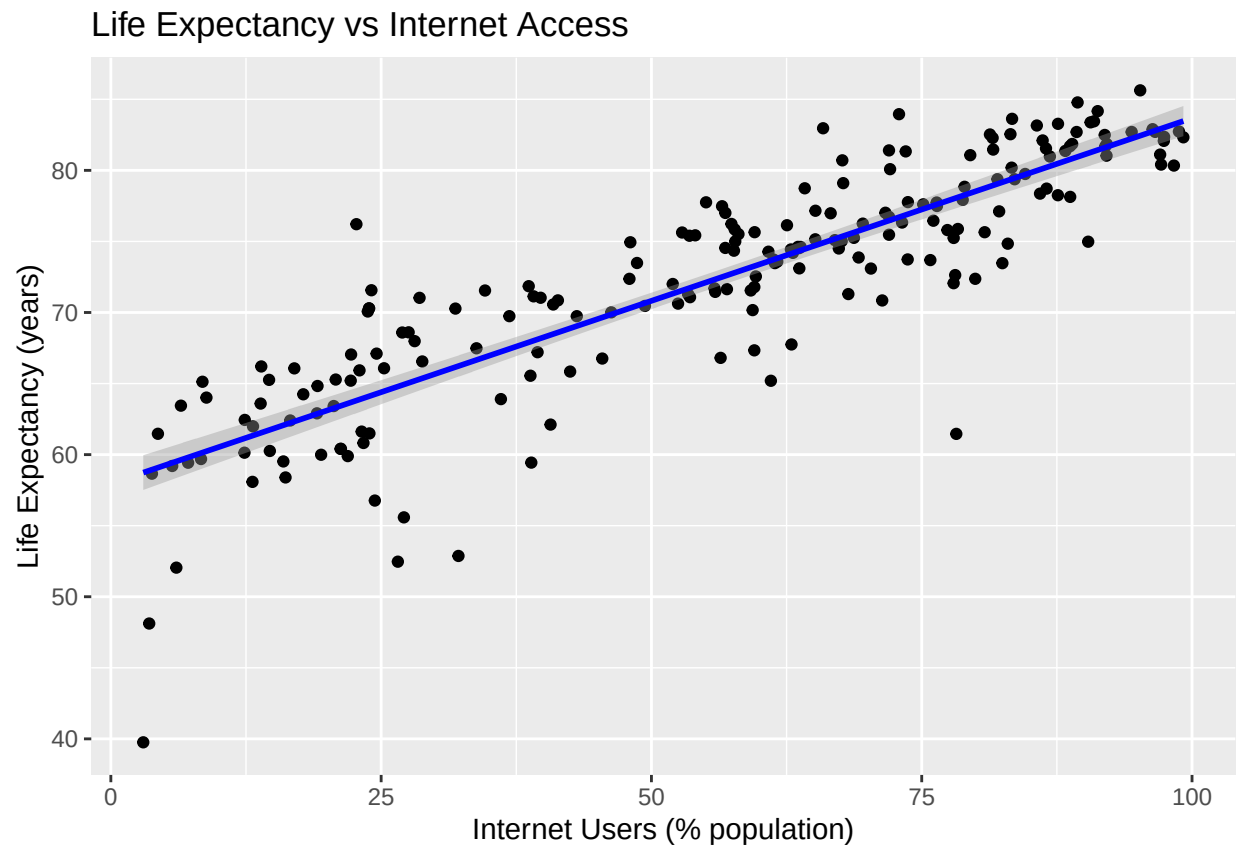
```
## 1st Qu.: 64.71    1st Qu.: 2010    1st Qu.:66.77    1st Qu.: 7.606
## Median : 89.50    Median : 6249    Median :73.71    Median : 8.740
## Mean   : 84.01    Mean   : 16438   Mean   :72.26    Mean   : 8.777
## 3rd Qu.:102.24    3rd Qu.: 18520   3rd Qu.:78.08    3rd Qu.: 9.827
## Max.   :158.05    Max.   :173605   Max.   :85.63    Max.   :12.065
## internet_c      school_c      log_gdp_c
## Min.   :-52.598   Min.   :-72.479   Min.   :-3.32132
## 1st Qu.: -27.385   1st Qu.: -19.300   1st Qu.: -1.17041
## Median :  4.002    Median :  5.491    Median : -0.03644
## Mean   :  0.000    Mean   :  0.000    Mean   :  0.00000
## 3rd Qu.: 22.709    3rd Qu.: 18.224    3rd Qu.:  1.04994
## Max.   : 43.602    Max.   : 74.038    Max.   :  3.28783
```

Exploratory Visualizations

Internet vs Life Expectancy

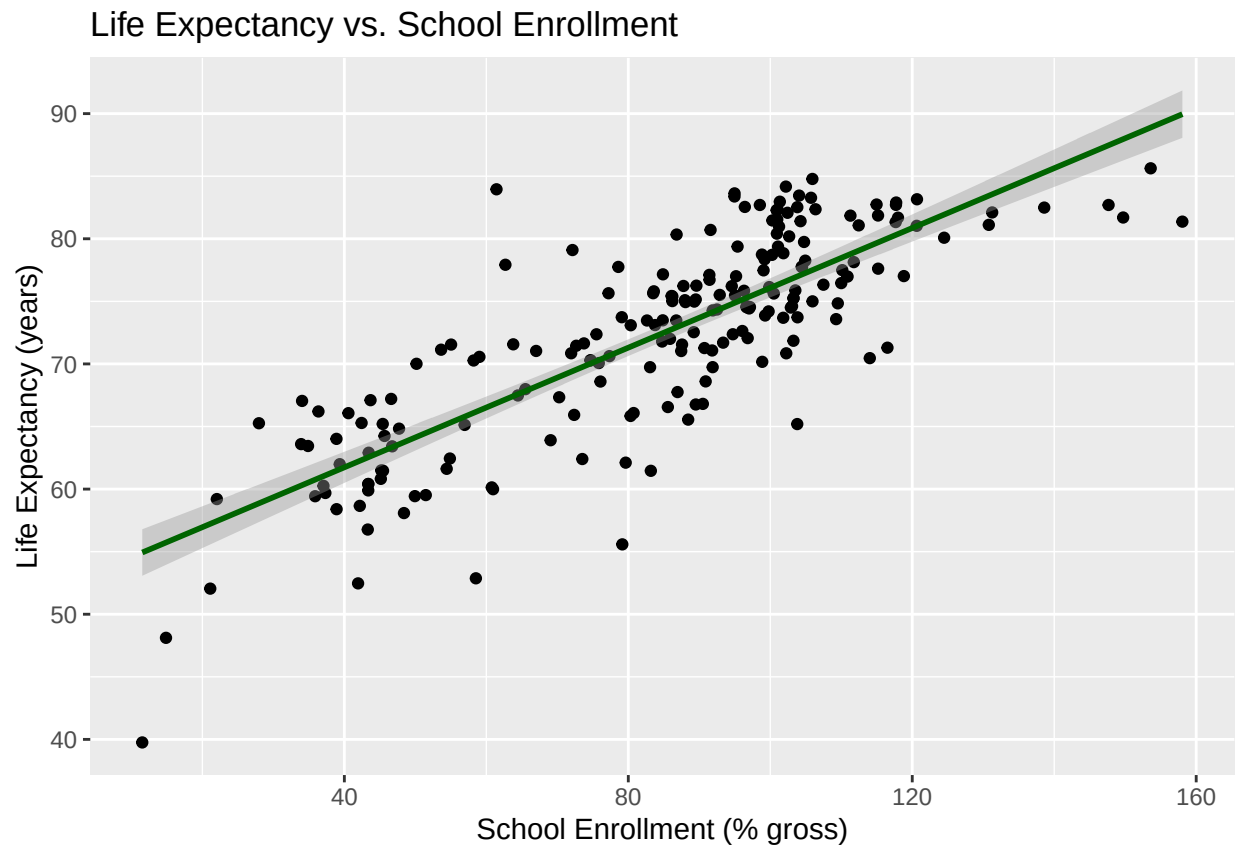
```
ggplot(cleaned, aes(x = internet, y = lifeexp)) +
  geom_point() +
  geom_smooth(method = "lm", color = "blue") +
  labs(title = "Life Expectancy vs Internet Access",
       x = "Internet Users (% population)",
       y = "Life Expectancy (years)")
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```



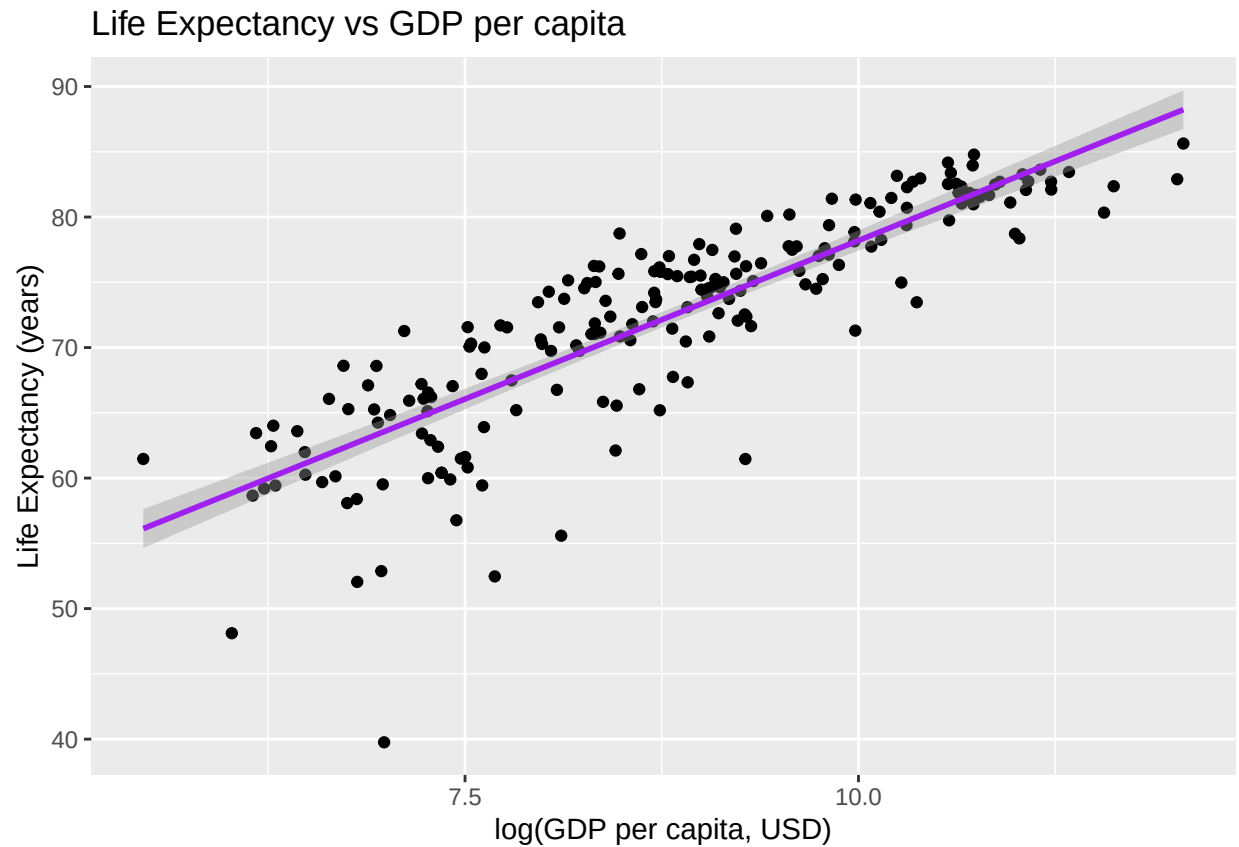
```
## School Enrollment vs Life Expectancy
ggplot(cleaned, aes(x = school_enroll, y = lifeexp)) +
  geom_point() +
  geom_smooth(method = "lm", color = "darkgreen") +
  labs(title = "Life Expectancy vs. School Enrollment",
       x = "School Enrollment (% gross)",
       y = "Life Expectancy (years)")
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```



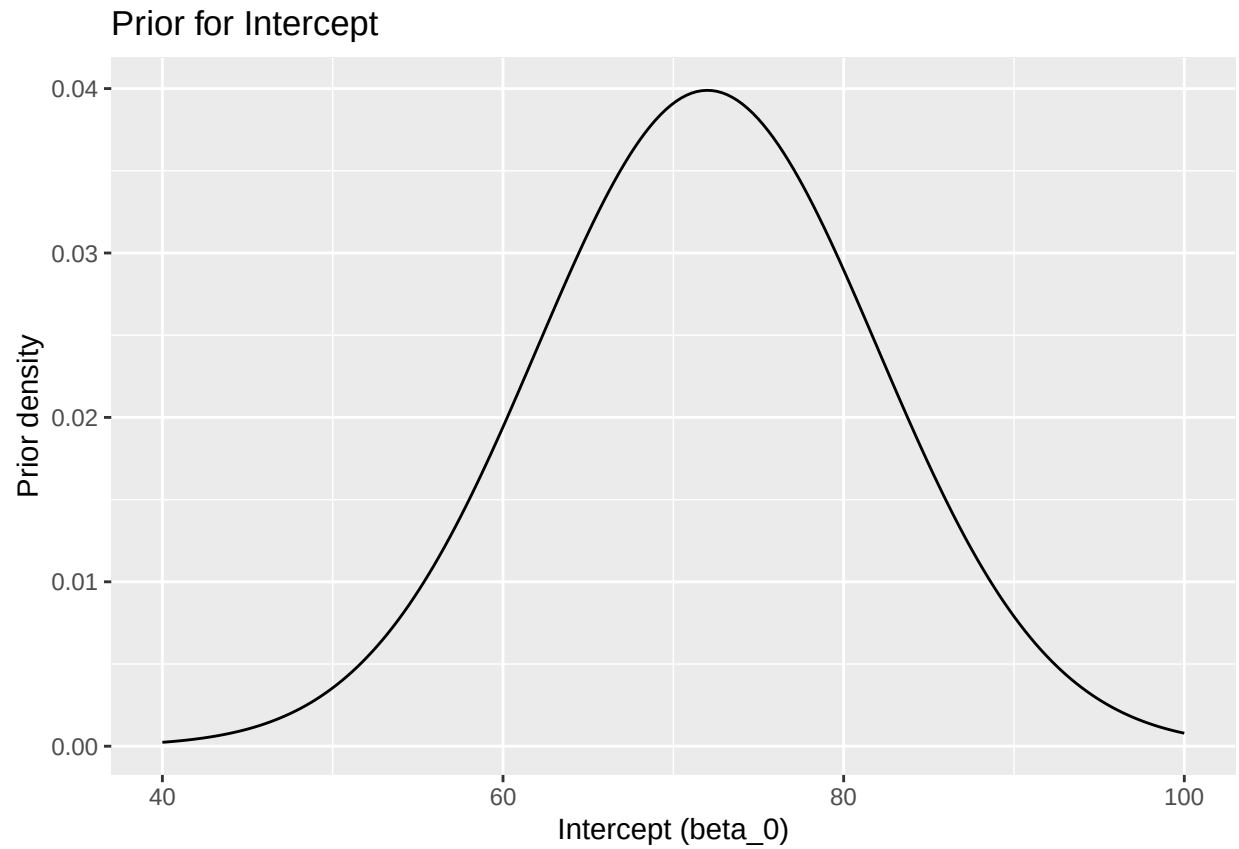
```
## Log GDP vs Life Expectancy
ggplot(cleaned, aes(x = log_gdp, y = lifeexp)) +
  geom_point() +
  geom_smooth(method = "lm", color = "purple") +
  labs(title = "Life Expectancy vs GDP per capita",
       x = "log(GDP per capita, USD)",
       y = "Life Expectancy (years)")
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

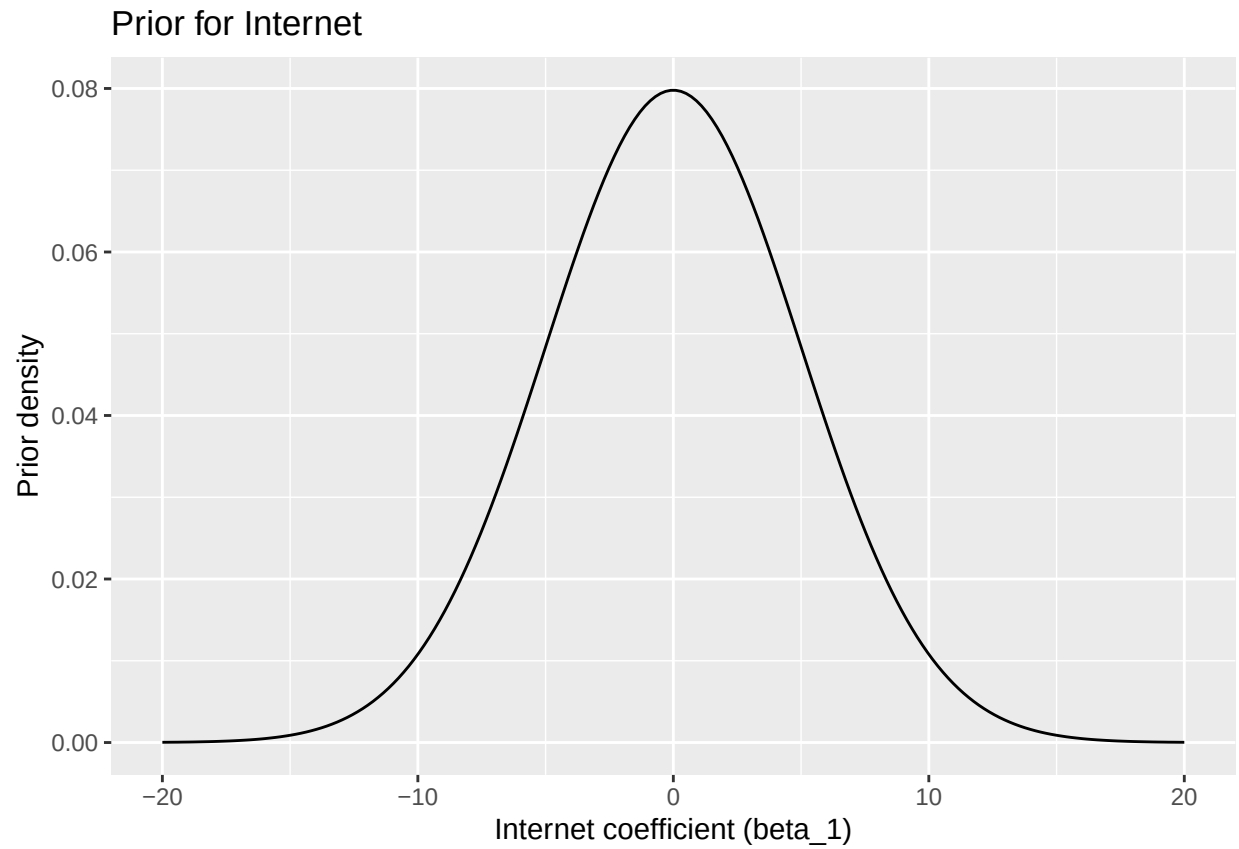


```
# Specify Priors

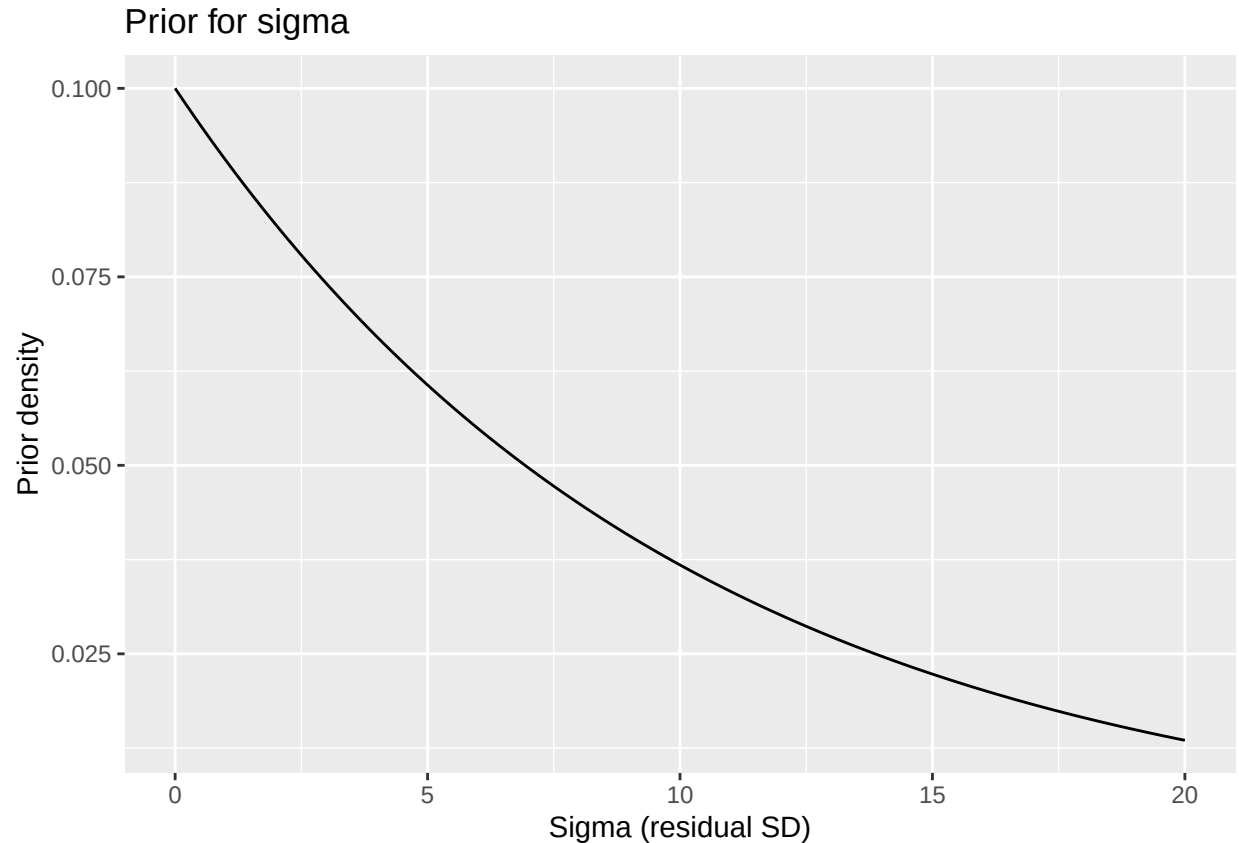
# Intercept prior
x0 <- seq(40, 100, length.out = 200)
y0 <- dnorm(x0, mean = 72, sd = 10)
ggplot(data.frame(x0, y0), aes(x = x0, y = y0)) +
  geom_line() +
  labs(x = "Intercept (beta_0)", y = "Prior density", title = "Prior for Intercept")
```



```
# Sigma prior (same for rest of predictors)
x1 <- seq(-20, 20, length.out = 200)
y1 <- dnorm(x1, mean = 0, sd = 5)
ggplot(data.frame(x1, y1), aes(x = x1, y = y1)) +
  geom_line() +
  labs(x = "Internet coefficient (beta_1)", y = "Prior density", title = "Prior for Internet")
```



```
# Sigma prior
x_sigma <- seq(0, 20, length.out = 200)
y_sigma <- dexp(x_sigma, rate = 0.1)
ggplot(data.frame(x_sigma, y_sigma), aes(x = x_sigma, y = y_sigma)) +
  geom_line() +
  labs(x = "Sigma (residual SD)", y = "Prior density", title = "Prior for sigma")
```



Fit Bayesian linear regression using rstanarm (from 9.3.1 of Bayes Rule)

```
set.seed(123)
wdi_model <- stan_glm(
  lifeexp ~ internet_c + school_c + log_gdp_c,
  data = cleaned,
  family = gaussian(),
  prior_intercept = normal(72, 10),
  prior = normal(0, 5),
  prior_aux = exponential(0.1), # for sigma
  chains = 4, iter = 2000
)
```

```
##
## SAMPLING FOR MODEL 'continuous' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 4.8e-05 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.48 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: Iteration:    1 / 2000 [  0%] (Warmup)
## Chain 1: Iteration:  200 / 2000 [ 10%] (Warmup)
## Chain 1: Iteration:  400 / 2000 [ 20%] (Warmup)
## Chain 1: Iteration:  600 / 2000 [ 30%] (Warmup)
## Chain 1: Iteration:  800 / 2000 [ 40%] (Warmup)
```

```

## Chain 1: Iteration: 1000 / 2000 [ 50%] (Warmup)
## Chain 1: Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 1: Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 1: Iteration: 1400 / 2000 [ 70%] (Sampling)
## Chain 1: Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 1: Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 1: Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 1:
## Chain 1: Elapsed Time: 0.106 seconds (Warm-up)
## Chain 1: 0.104 seconds (Sampling)
## Chain 1: 0.21 seconds (Total)
## Chain 1:
##
## SAMPLING FOR MODEL 'continuous' NOW (CHAIN 2).
## Chain 2:
## Chain 2: Gradient evaluation took 9e-06 seconds
## Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.09 seconds.
## Chain 2: Adjust your expectations accordingly!
## Chain 2:
## Chain 2:
## Chain 2: Iteration: 1 / 2000 [ 0%] (Warmup)
## Chain 2: Iteration: 200 / 2000 [ 10%] (Warmup)
## Chain 2: Iteration: 400 / 2000 [ 20%] (Warmup)
## Chain 2: Iteration: 600 / 2000 [ 30%] (Warmup)
## Chain 2: Iteration: 800 / 2000 [ 40%] (Warmup)
## Chain 2: Iteration: 1000 / 2000 [ 50%] (Warmup)
## Chain 2: Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 2: Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 2: Iteration: 1400 / 2000 [ 70%] (Sampling)
## Chain 2: Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 2: Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 2: Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 2:
## Chain 2: Elapsed Time: 0.107 seconds (Warm-up)
## Chain 2: 0.092 seconds (Sampling)
## Chain 2: 0.199 seconds (Total)
## Chain 2:
##
## SAMPLING FOR MODEL 'continuous' NOW (CHAIN 3).
## Chain 3:
## Chain 3: Gradient evaluation took 1.1e-05 seconds
## Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.11 seconds.
## Chain 3: Adjust your expectations accordingly!
## Chain 3:
## Chain 3:
## Chain 3: Iteration: 1 / 2000 [ 0%] (Warmup)
## Chain 3: Iteration: 200 / 2000 [ 10%] (Warmup)
## Chain 3: Iteration: 400 / 2000 [ 20%] (Warmup)
## Chain 3: Iteration: 600 / 2000 [ 30%] (Warmup)
## Chain 3: Iteration: 800 / 2000 [ 40%] (Warmup)
## Chain 3: Iteration: 1000 / 2000 [ 50%] (Warmup)
## Chain 3: Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 3: Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 3: Iteration: 1400 / 2000 [ 70%] (Sampling)

```

```

## Chain 3: Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 3: Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 3: Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 3:
## Chain 3: Elapsed Time: 0.108 seconds (Warm-up)
## Chain 3: 0.098 seconds (Sampling)
## Chain 3: 0.206 seconds (Total)
## Chain 3:
##
## SAMPLING FOR MODEL 'continuous' NOW (CHAIN 4).
## Chain 4:
## Chain 4: Gradient evaluation took 9e-06 seconds
## Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.09 seconds.
## Chain 4: Adjust your expectations accordingly!
## Chain 4:
## Chain 4:
## Chain 4: Iteration: 1 / 2000 [ 0%] (Warmup)
## Chain 4: Iteration: 200 / 2000 [ 10%] (Warmup)
## Chain 4: Iteration: 400 / 2000 [ 20%] (Warmup)
## Chain 4: Iteration: 600 / 2000 [ 30%] (Warmup)
## Chain 4: Iteration: 800 / 2000 [ 40%] (Warmup)
## Chain 4: Iteration: 1000 / 2000 [ 50%] (Warmup)
## Chain 4: Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 4: Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 4: Iteration: 1400 / 2000 [ 70%] (Sampling)
## Chain 4: Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 4: Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 4: Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 4:
## Chain 4: Elapsed Time: 0.113 seconds (Warm-up)
## Chain 4: 0.096 seconds (Sampling)
## Chain 4: 0.209 seconds (Total)
## Chain 4:

```

Convergence Diagnostics

```
neff_ratio(wdi_model)
```

```

## (Intercept) internet_c school_c log_gdp_c sigma
## 0.7410 0.5695 0.9265 0.5435 0.6800

```

```
rhat(wdi_model)
```

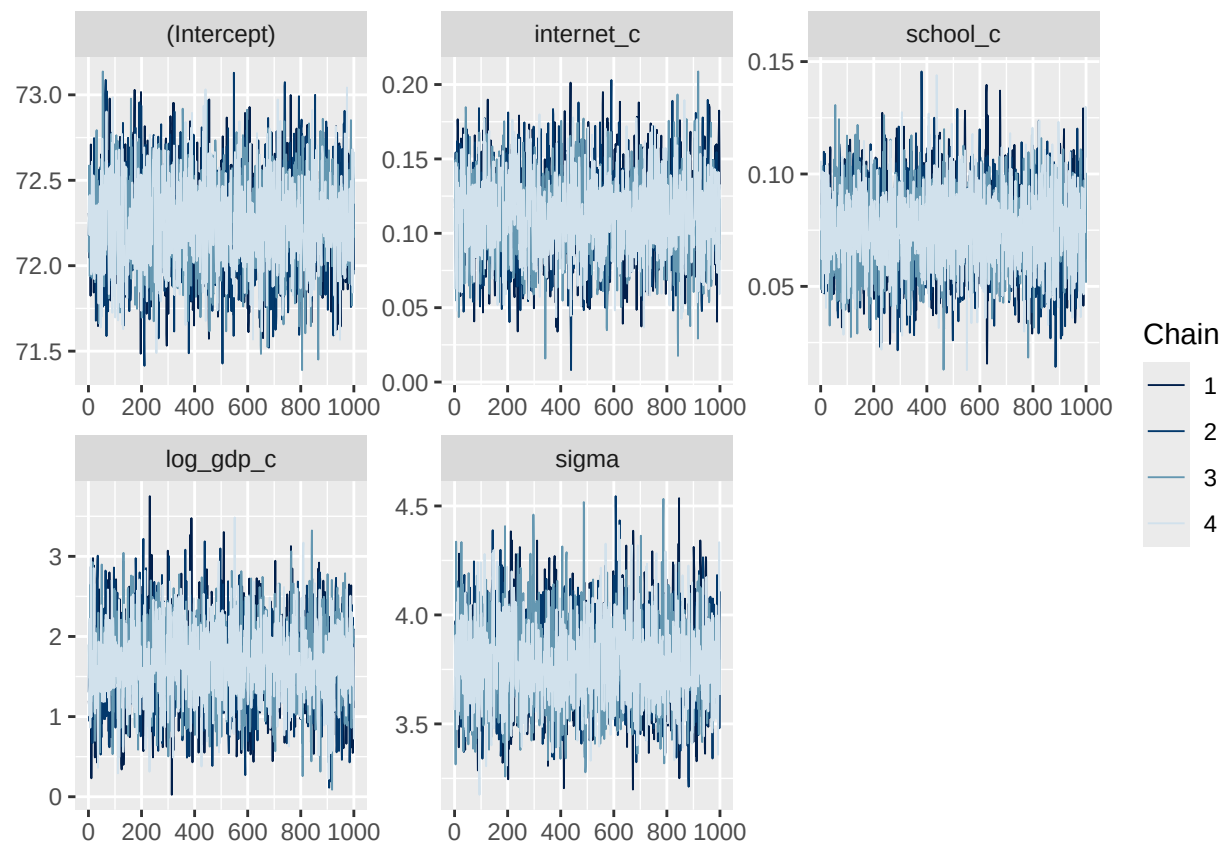
```

## (Intercept) internet_c school_c log_gdp_c sigma
## 1.0002375 0.9995410 0.9999039 0.9998164 1.0001642

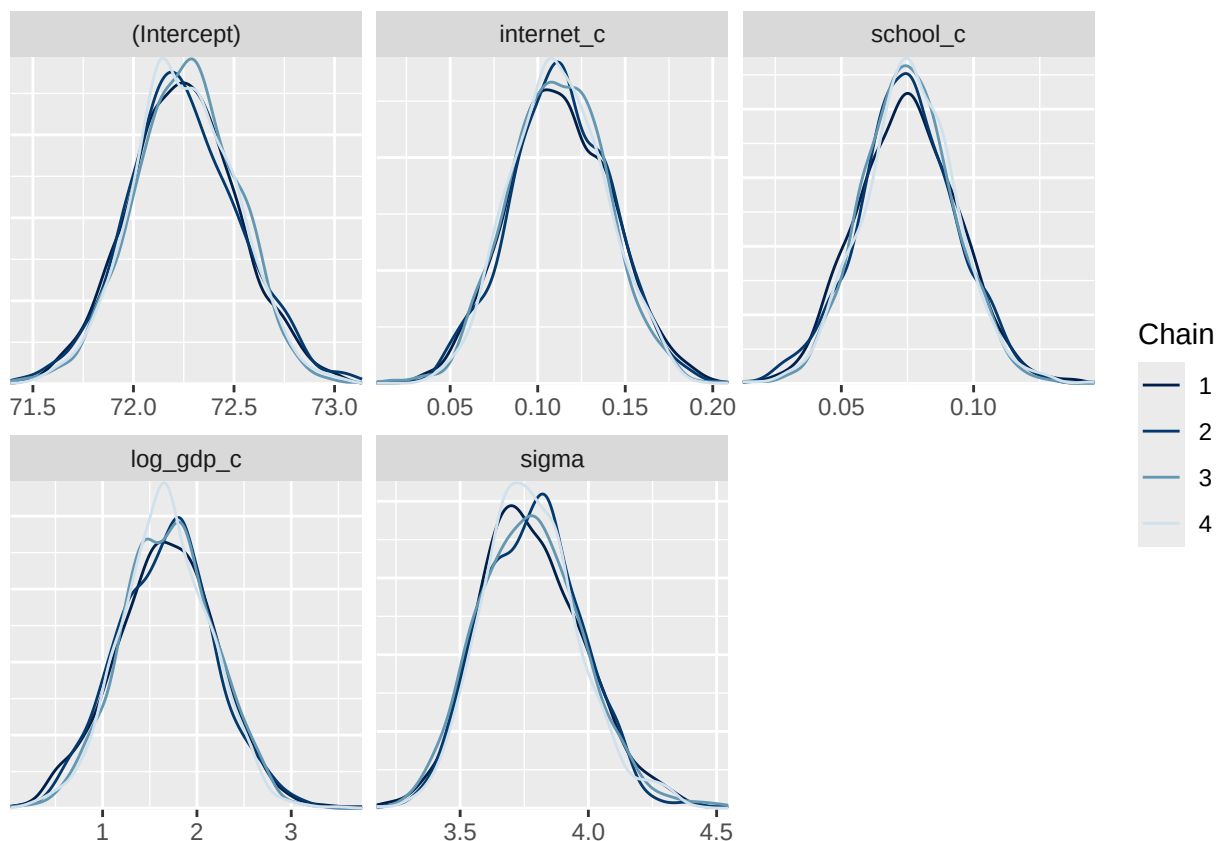
```

Trace plots

```
mcmc_trace(as.array(wdi_model))
```



```
# Density overlays
mcmc_dens_overlay(as.array(wdi_model))
```



Posterior Summaries

```
posterior_summary <- tidy_draws(wdi_model) %>%
  median_qi(.width = 0.95)
```

```
posterior_summary
```

```
## # A tibble: 1 x 36
##   '(Intercept)' '(Intercept).lower' '(Intercept).upper' internet_c
##   <dbl>          <dbl>          <dbl>          <dbl>
## 1      72.2          71.7          72.8          0.112
## # i 32 more variables: internet_c.lower <dbl>, internet_c.upper <dbl>,
## #   school_c <dbl>, school_c.lower <dbl>, school_c.upper <dbl>,
## #   log_gdp_c <dbl>, log_gdp_c.lower <dbl>, log_gdp_c.upper <dbl>, sigma <dbl>,
## #   sigma.lower <dbl>, sigma.upper <dbl>, accept_stat__ <dbl>,
## #   accept_stat__.lower <dbl>, accept_stat__.upper <dbl>, stepsize__ <dbl>,
## #   stepsize__.lower <dbl>, stepsize__.upper <dbl>, treedepth__ <dbl>,
## #   treedepth__.lower <dbl>, treedepth__.upper <dbl>, n_leapfrog__ <dbl>, ...
```

```
posterior_draws <- wdi_model %>%
  tidy_draws() %>%
  select(`(Intercept)`, internet_c, school_c, log_gdp_c, sigma)
```

```
posterior_probs <- posterior_draws %>%
  summarize(
```

```

p_intercept_pos = mean(`(Intercept)` > 0),
p_internet_pos  = mean(internet_c > 0),
p_school_pos    = mean(school_c > 0),
p_gdp_pos       = mean(log_gdp_c > 0),
p_sigma_pos     = mean(sigma > 0)
)

```

```
posterior_probs
```

```

## # A tibble: 1 x 5
##   p_intercept_pos p_internet_pos p_school_pos p_gdp_pos p_sigma_pos
##           <dbl>           <dbl>           <dbl>         <dbl>         <dbl>
## 1             1             1             1             1             1

```

```
# Posterior Fitted Values
```

```
set.seed(123)
```

```

posterior_fits <- cleaned %>%
  add_fitted_draws(wdi_model, n = 50)

```

```

## Warning in fitted_draws.default(model = model, newdata = newdata, ..., n = n): 'fitted_draws' and 'a
## - Use [add_]epred_draws() to get the expectation of the posterior predictive.
## - Use [add_]linpred_draws() to get the distribution of the linear predictor.
## - For example, you used [add_]fitted_draws(..., scale = "response"), which
##   means you most likely want [add_]epred_draws(...).
## NOTE: When updating to the new functions, note that the 'model' parameter is now
##   named 'object' and the 'n' parameter is now named 'ndraws'.

```

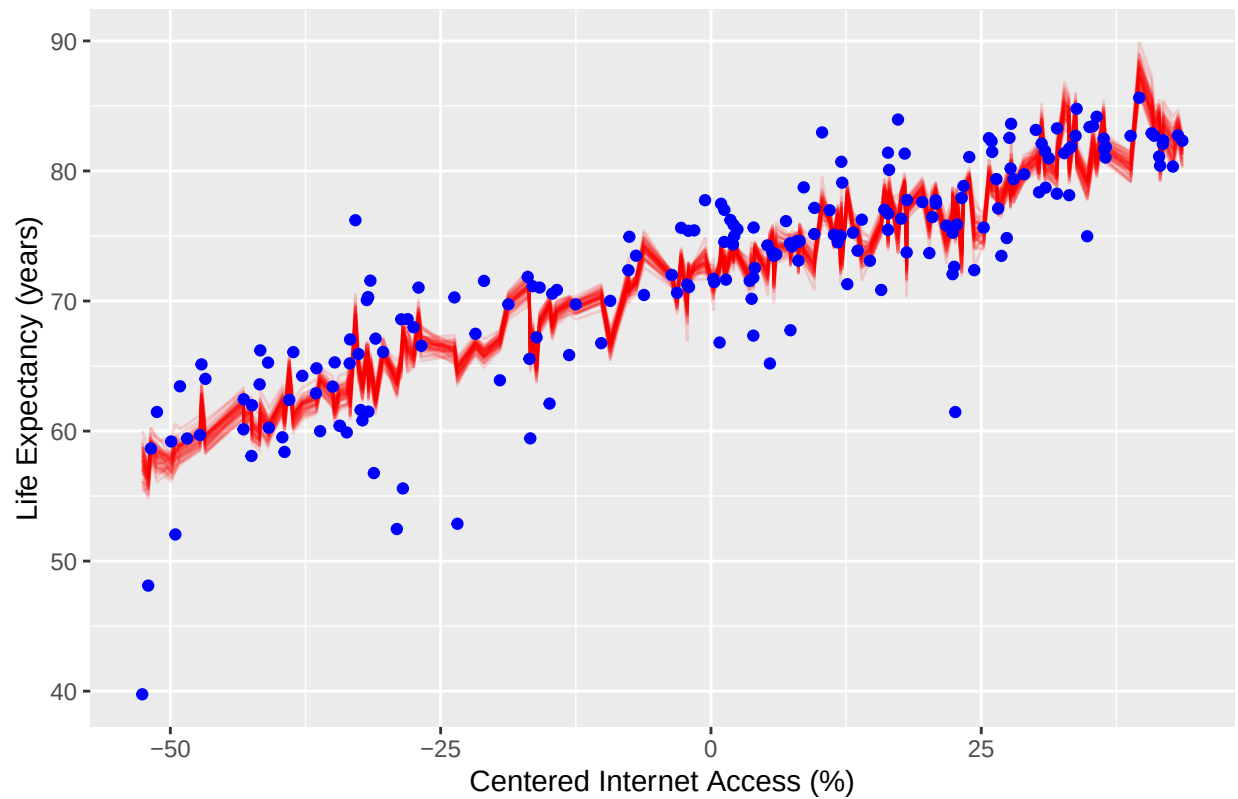
```
# Internet Access vs Life Expectancy
```

```

ggplot(posterior_fits, aes(x = internet_c, y = .value)) +
  geom_line(aes(group = .draw), alpha = 0.15, color = "red") + # 50 model lines
  geom_point(data = cleaned, aes(y = lifeexp), color = "blue") + # actual data points
  labs(
    title = "Posterior Fitted Lines: Internet Access vs Life Expectancy",
    x = "Centered Internet Access (%)",
    y = "Life Expectancy (years)"
  )

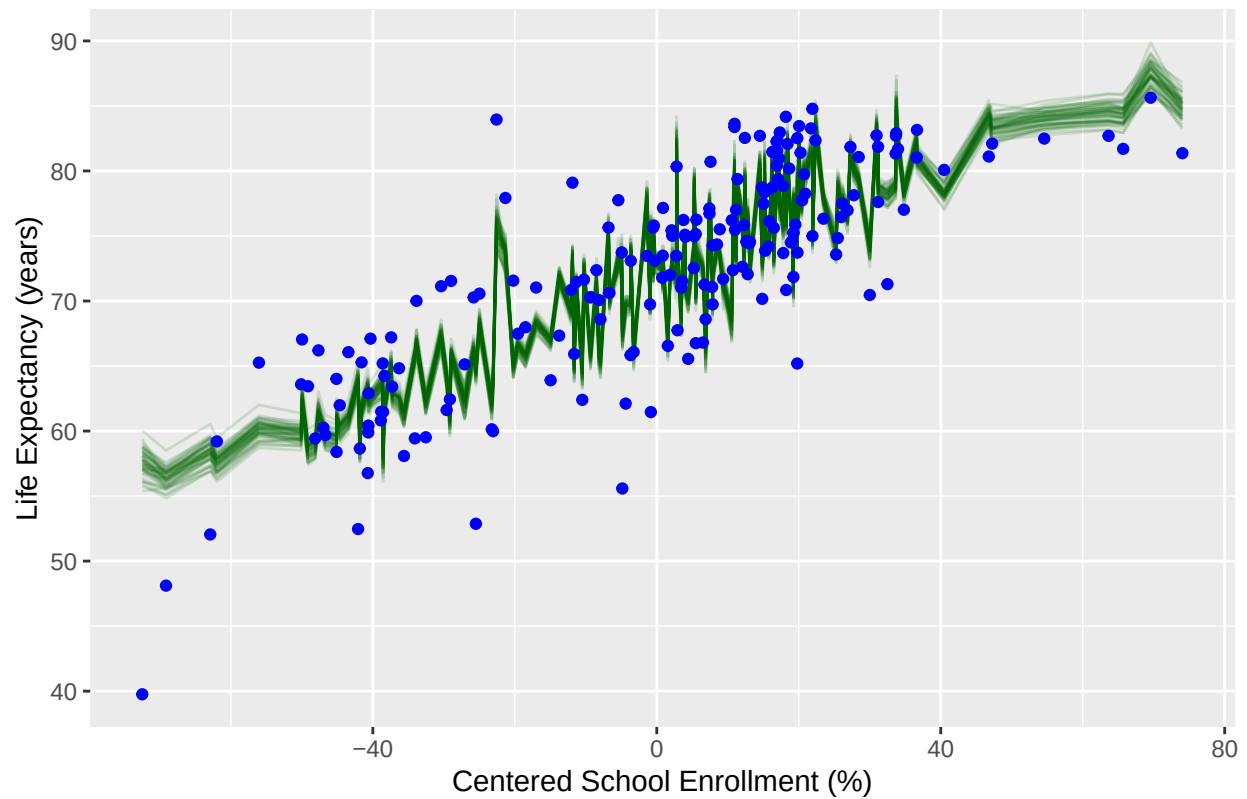
```

Posterior Fitted Lines: Internet Access vs Life Expectancy



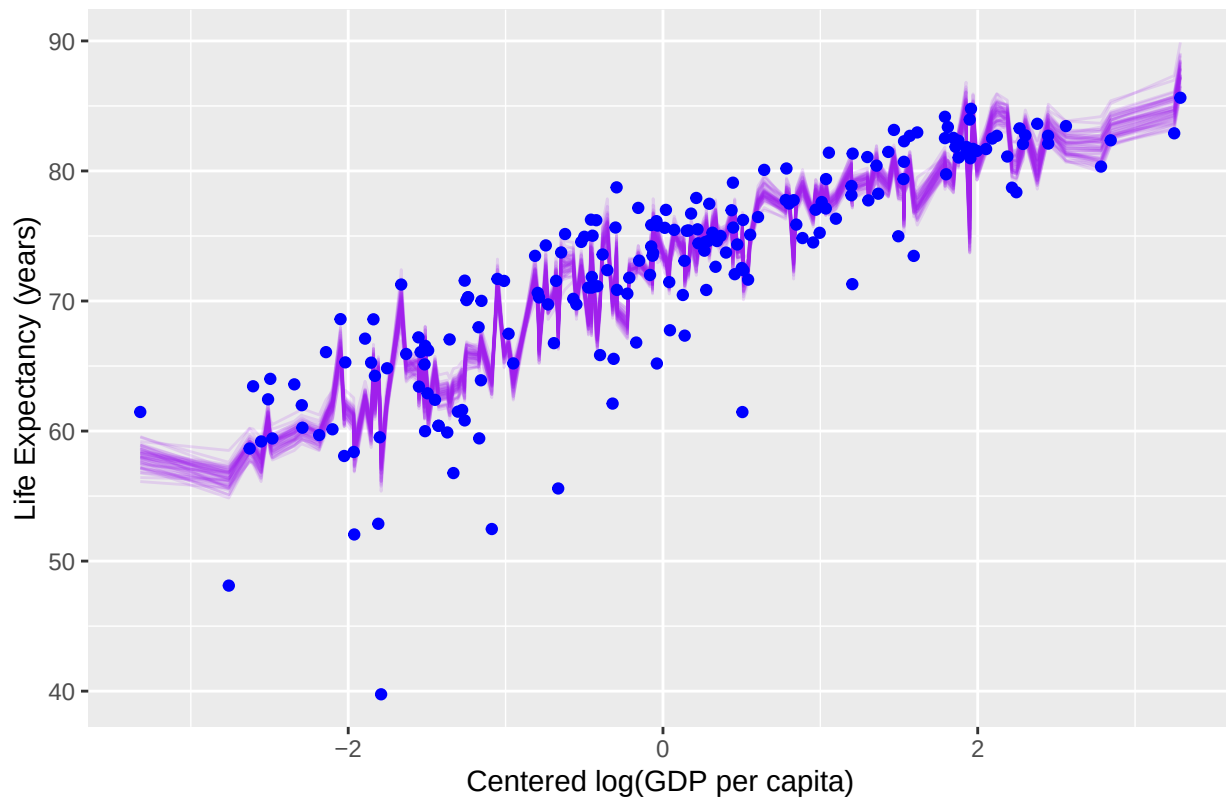
```
# School Enrollment vs Life Expectancy
ggplot(posterior_fits, aes(x = school_c, y = .value)) +
  geom_line(aes(group = .draw), alpha = 0.15, color = "darkgreen") +
  geom_point(data = cleaned, aes(y = lifeexp), color = "blue") +
  labs(
    title = "Posterior Fitted Lines: School Enrollment vs Life Expectancy",
    x = "Centered School Enrollment (%)",
    y = "Life Expectancy (years)"
  )
```

Posterior Fitted Lines: School Enrollment vs Life Expectancy



```
# Log GDP vs Life Expectancy
ggplot(posterior_fits, aes(x = log_gdp_c, y = .value)) +
  geom_line(aes(group = .draw), alpha = 0.15, color = "purple") +
  geom_point(data = cleaned, aes(y = lifeexp), color = "blue") +
  labs(
    title = "Posterior Fitted Lines: Log GDP vs Life Expectancy",
    x = "Centered log(GDP per capita)",
    y = "Life Expectancy (years)"
  )
```

Posterior Fitted Lines: Log GDP vs Life Expectancy



```
# Posterior Predictive Simulations

## Define scenarios
posterior_draws <- tidy_draws(wdi_model)

## Rename parameters to avoid conflicts with scenario variables
posterior_draws <- posterior_draws %>%
  rename(
    b_Intercept = `(Intercept)`,
    b_internet_c = internet_c,
    b_school_c   = school_c,
    b_log_gdp_c  = log_gdp_c,
    b_sigma      = sigma
  )

## Scenario definitions (low, average, high values)
new_countries <- tibble(
  internet_c = c(-20, 0, 20),
  school_c   = c(-10, 0, 10),
  log_gdp_c  = c(-1, 0, 1)
) %>%
  mutate(
    scenario = paste0(
      "Internet: ", internet_c, ", ",
      "School: ", school_c, ", ",
      "logGDP: ", log_gdp_c
    )
  )
```

```

    )
  )

## Generate posterior predictions
set.seed(123)

posterior_preds <- posterior_draws %>%
  crossing(new_countries) %>%
  mutate(
    mu = b_Intercept +
      b_internet_c * internet_c +
      b_school_c * school_c +
      b_log_gdp_c * log_gdp_c,
    y_new = rnorm(n(), mean = mu, sd = b_sigma)
  )

## Summaries per scenario
posterior_summary <- posterior_preds %>%
  group_by(scenario) %>%
  summarize(
    mean_mu = mean(mu),
    low_mu = quantile(mu, 0.025),
    high_mu = quantile(mu, 0.975),
    mean_y = mean(y_new),
    low_y = quantile(y_new, 0.025),
    high_y = quantile(y_new, 0.975),
    .groups = "drop"
  )

posterior_summary

```

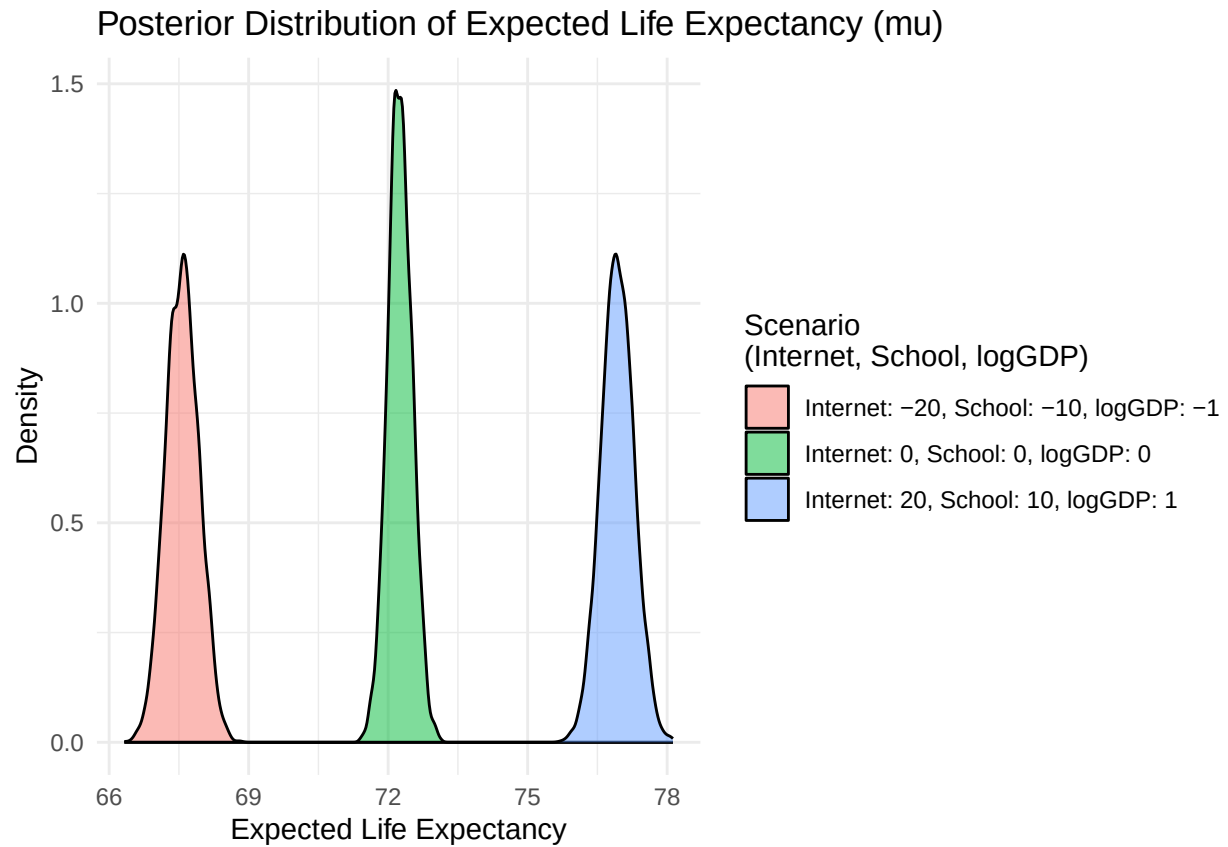
```
## # A tibble: 3 x 7
```

## scenario	mean_mu	low_mu	high_mu	mean_y	low_y	high_y
## <chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
## 1 Internet: -20, School: -10, logGDP~	67.6	66.9	68.3	67.5	59.9	75.0
## 2 Internet: 0, School: 0, logGDP: 0	72.3	71.7	72.8	72.3	64.9	79.7
## 3 Internet: 20, School: 10, logGDP: 1	76.9	76.2	77.6	77.0	69.6	84.4

Posterior predictive plots

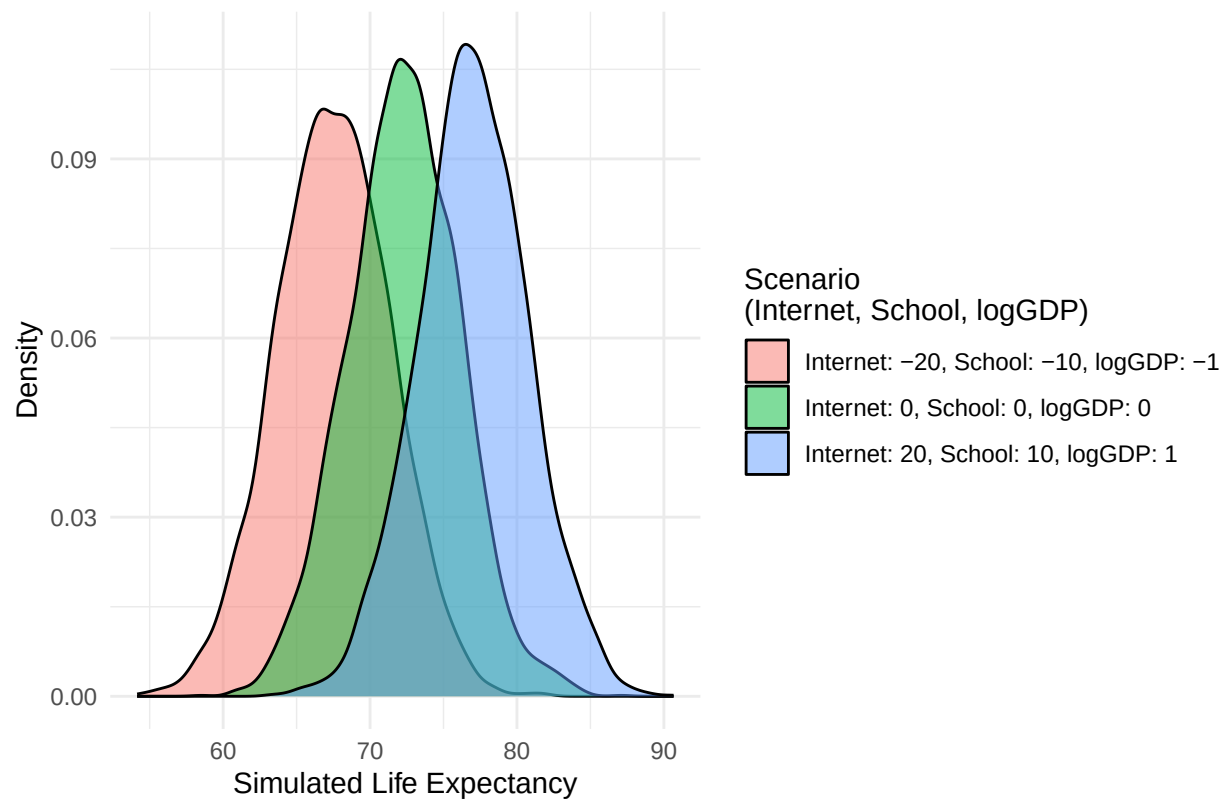
```
## Plot 1: Expected life expectancy (mu)
ggplot(posterior_preds, aes(x = mu, fill = scenario)) +
  geom_density(alpha = 0.5) +
  labs(
    title = "Posterior Distribution of Expected Life Expectancy (mu)",
    x = "Expected Life Expectancy",
    y = "Density",
    fill = "Scenario\n(Internet, School, logGDP)"
  ) +
  theme_minimal()

```



```
# Plot 2: Simulated observed life expectancy (y_new)
ggplot(posterior_preds, aes(x = y_new, fill = scenario)) +
  geom_density(alpha = 0.5) +
  labs(
    title = "Posterior Predictive Distribution of Life Expectancy (y_new)",
    x = "Simulated Life Expectancy",
    y = "Density",
    fill = "Scenario\n(Internet, School, logGDP)"
  ) +
  theme_minimal()
```

Posterior Predictive Distribution of Life Expectancy (y_new)

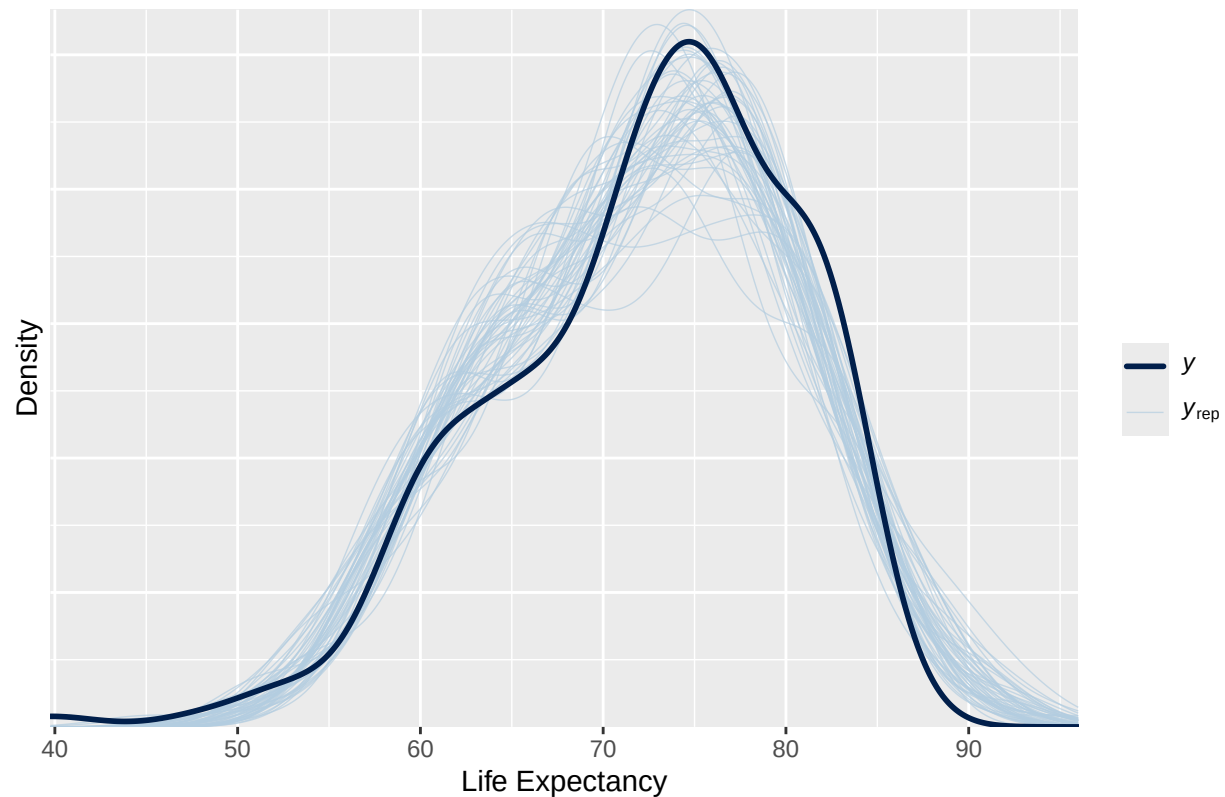


```
## Posterior Predictive Checks
```

```
pp_check(wdi_model, type = "dens_overlay", nreps = 50) +  
  labs(  
    title = "Posterior Predictive Check: Observed vs Simulated Life Expectancy",  
    x = "Life Expectancy",  
    y = "Density")
```

```
## Warning: The following arguments were unrecognized and ignored: type
```

Posterior Predictive Check: Observed vs Simulated Life Expectancy

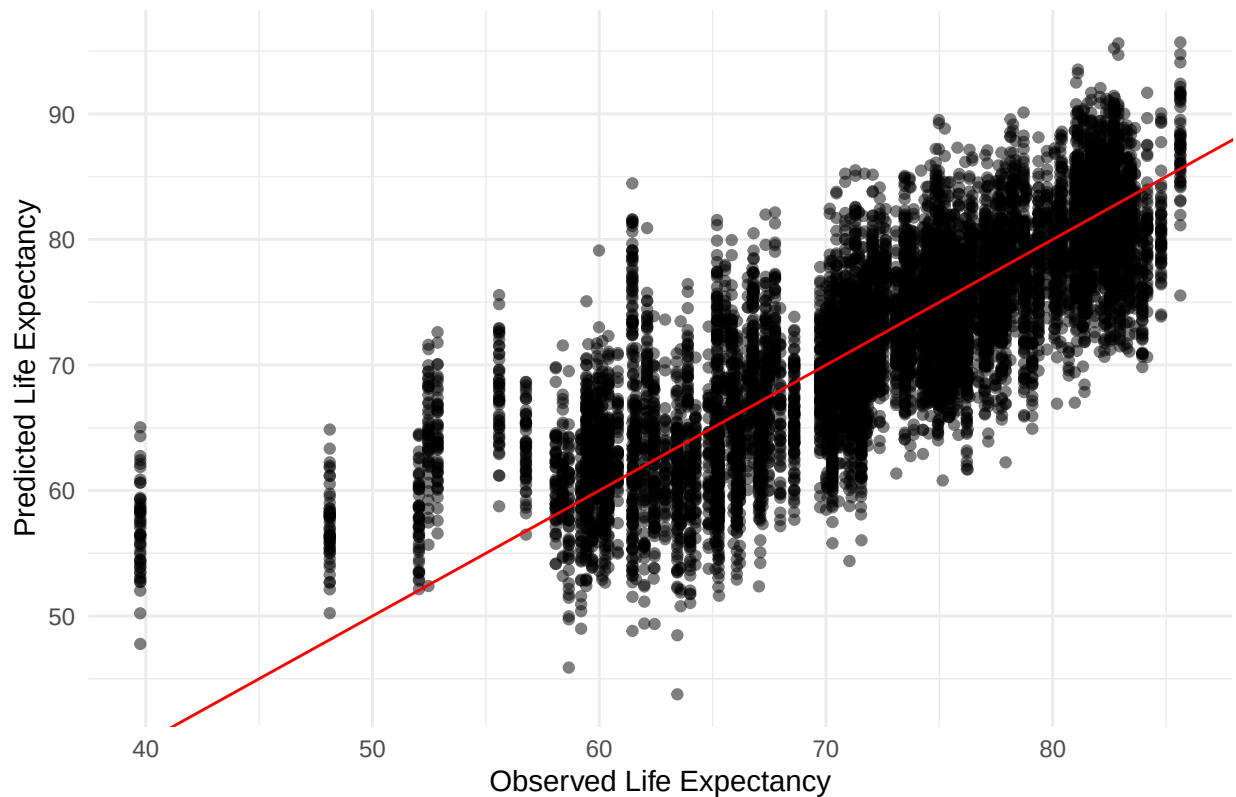


```
posterior_preds_obs <- add_predicted_draws(cleaned, wdi_model, n = 50)
```

```
## Warning:  
## In add_predicted_draws(): The 'n' argument is a deprecated alias for 'ndraws'.  
## Use the 'ndraws' argument instead.  
## See help("tidybayes-deprecated").
```

```
ggplot(posterior_preds_obs, aes(y = .prediction, x = lifeexp)) +  
  geom_point(alpha = 0.5) +  
  geom_abline(slope = 1, intercept = 0, color = "red") +  
  labs(  
    title = "Observed vs Posterior Predicted Life Expectancy",  
    x = "Observed Life Expectancy",  
    y = "Predicted Life Expectancy"  
  ) +  
  theme_minimal()
```

Observed vs Posterior Predicted Life Expectancy



```
# Posterior Predictive Summaries
```

```
set.seed(123)
```

```
## Generate posterior predictive draws for all observations
```

```
predictions <- posterior_predict(wdi_model, newdata = cleaned)
```

```
## Quick check of dimensions: rows = posterior draws, cols = countries
```

```
dim(predictions)
```

```
## [1] 4000 202
```

```
## Visualize posterior predictive intervals for all countries
```

```
ppc_intervals(y = cleaned$lifeexp, yrep = predictions, x = 1:nrow(cleaned),  
              prob = 0.5, prob_outer = 0.95) +
```

```
  labs(
```

```
    x = "Country index",
```

```
    y = "Life Expectancy",
```

```
    title = "Posterior Predictive Intervals for Observed Life Expectancy"
```

```
)
```

```
## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
```

```
## i Please use 'linewidth' instead.
```

```
## i The deprecated feature was likely used in the bayesplot package.
```

```
## Please report the issue at <https://github.com/stan-dev/bayesplot/issues/>.
```

```
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```

Posterior Predictive Intervals for Observed Life Expectancy



```
# Numerical posterior predictive summaries
set.seed(123)
prediction_summary(wdi_model, data = cleaned)
```

```
##           mae mae_scaled within_50 within_95
## 1 2.168674  0.5708601 0.5841584 0.9554455
```